

Compiladores

Aula 19: Otimização de laços e integração do compilador

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Dominância
- Invariantes
- Indução
- Integração do compilador MINIC

Dominância

- O percurso completo de compilação de um programa possui 8 fases.
- Cada fronteira de fase tem uma entrada, uma saída e um contrato verificável.



Falha detectada

Interrompe o avanço; não deixa erro léxico/sintático/semântico virar código enganoso.

Contrato explícito

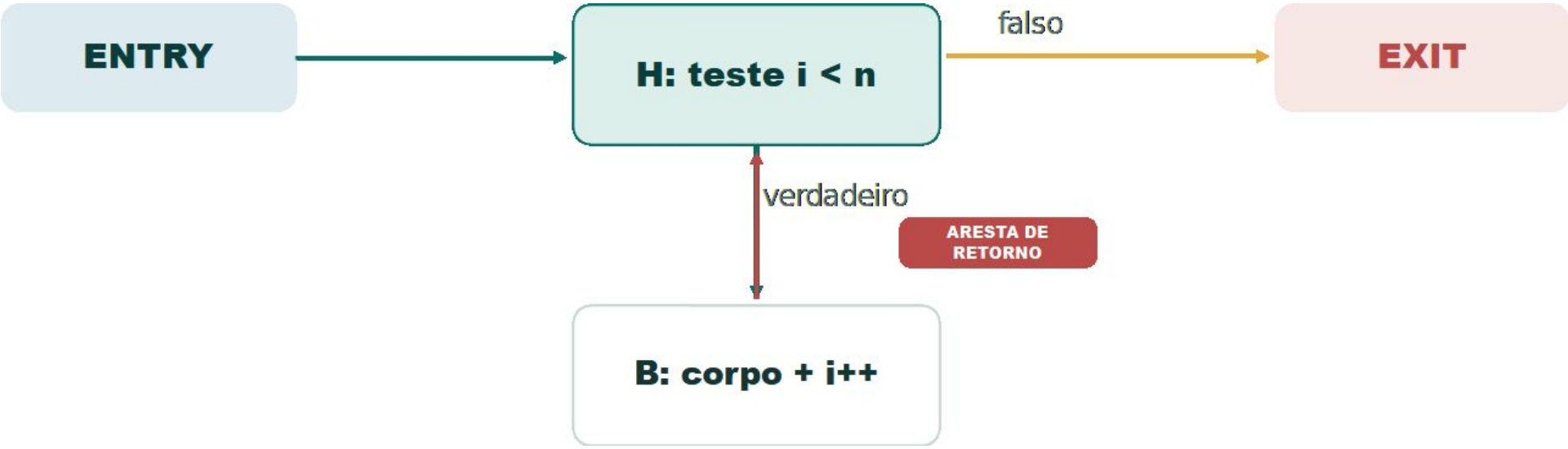
Formato e responsabilidades entre fases são estáveis e documentados.

Observável final

Código de saída, saída padrão e efeitos definidos pelo contrato.

Dominância

- Laços em grafos de fluxo:
  - O cabeçalho do laço é o ponto de teste visitado antes de cada iteração.



A forma explícita evidencia que o laço pode executar zero vezes.

Dominância

- Uma volta na busca em profundidade não basta: a condição é definida por dominância.

Dominância

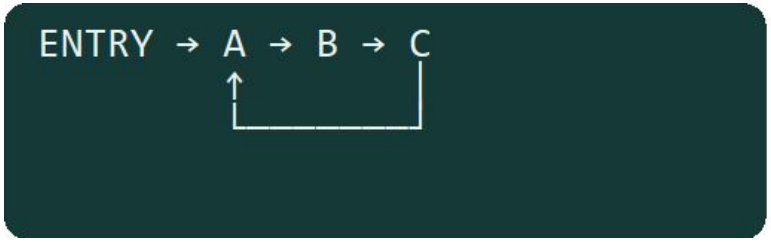
d domina n se todo caminho da entrada até n passa por d. Todo nó domina a si mesmo.

Back edge

Aresta a → h é de retorno quando h domina a. h é o cabeçalho; a é a cauda.

Laço natural

Inclui h e os predecessores que alcançam a sem atravessar h como novo nó.



B domina C ⇒ C → B é back edge ⇒ laço natural {B, C}

Invariantes

- Invariantes e movimentação de código:
  - A expressão deve permanecer estável e continuar sendo avaliada nos mesmos caminhos relevantes.

```
for (i = 0; i < n; i = i + 1) {
    dobro = limite * 2;
    total = total + dobro;
}
```



```
dobro = limite * 2;
for (i = 0; i < n; i = i + 1) {
    total = total + dobro;
}
```

Pode mover se...

limite não muda; não há chamada/escrita que o invalide; a expressão é pura e segura.

Cuidado com...

laço de zero iterações, exceções/erros, efeitos colaterais e semântica de overflow.

Invariantes

- É necessário observar que a invariância algébrica não garante equivalência observável.

ORIGINAL

```
while (condicao) {
    x = 10 / divisor;
    ...
}
```

Se condicao é falsa, a divisão não acontece.

MOVIDO

```
x = 10 / divisor;
while (condicao) {
    ...
}
```

Agora divisor = 0 pode falhar antes do teste.

Preservar quando e quantas vezes a expressão é avaliada faz parte da preservação semântica.

Indução

- Trocar recomputação por atualização incremental exige provar a relação em todas as iterações.

Indução básica

$i = i + 1$
Incremento constante por iteração.

Indução derivada

$j = 4 \times i + 3$
Relação afim com a básica.

Redução de força

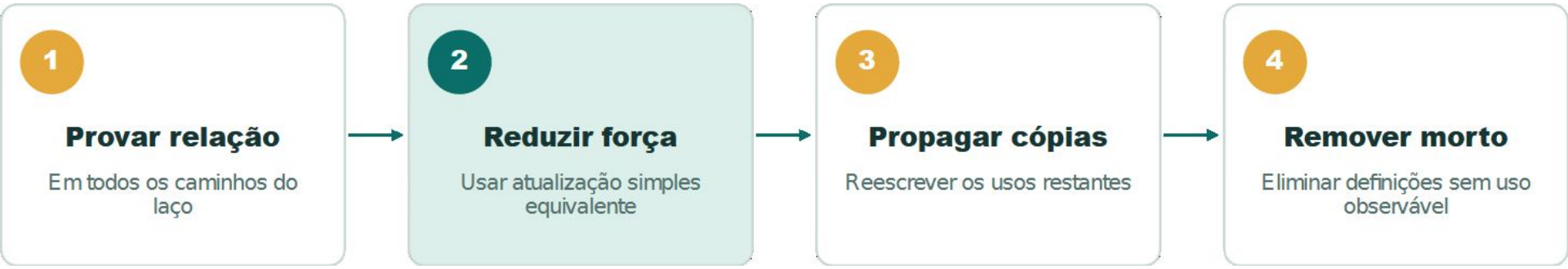
Trocar multiplicação repetida por soma incremental.

```
ANTES:   deslocamento = 4 * i;
DEPOIS:  deslocamento = 0;
         ...
         deslocamento = deslocamento + 4;
```

Validar limites, break/continue, passo e unidade do índice: MINIC indexa elementos, não bytes.

Indução

- Eliminação de variáveis de indução:
  - A simplificação vem depois da prova da relação e da redução de força.



Uma variável ainda necessária ou observável não pode ser apagada.

Indução

- Fusão de laços:
  - Reduz overhead e pode melhorar localidade;
  - a ordem de dependências precisa ser preservada.

Original:

```
for (i=0; i<n; i++) a[i] = b[i] + 1;
for (i=0; i<n; i++) c[i] = a[i] * 2;
```

Após fusão:

```
for (i=0; i<n; i++) {
    a[i] = b[i] + 1;
    c[i] = a[i] * 2;
}
```

Indução

- A fusão pode proporcionar os seguintes benefícios:
  - Menos controle de laço;
  - melhor proximidade de dados.

- Porém, necessita de uma prova:
  - Alias/overlap, limites iguais e dependências entre iterações.

Uma redução global no segundo laço pode depender de todo o primeiro: fusão pode mudar o resultado.

Indução

- Outra técnica possível é o desenrolamento, porém esta causa menos controle e mais corpo.
- Duplicar operações reduz testes e saltos, mas cresce o código e exige tratar o resto.

Fator de desenrolamento = 2

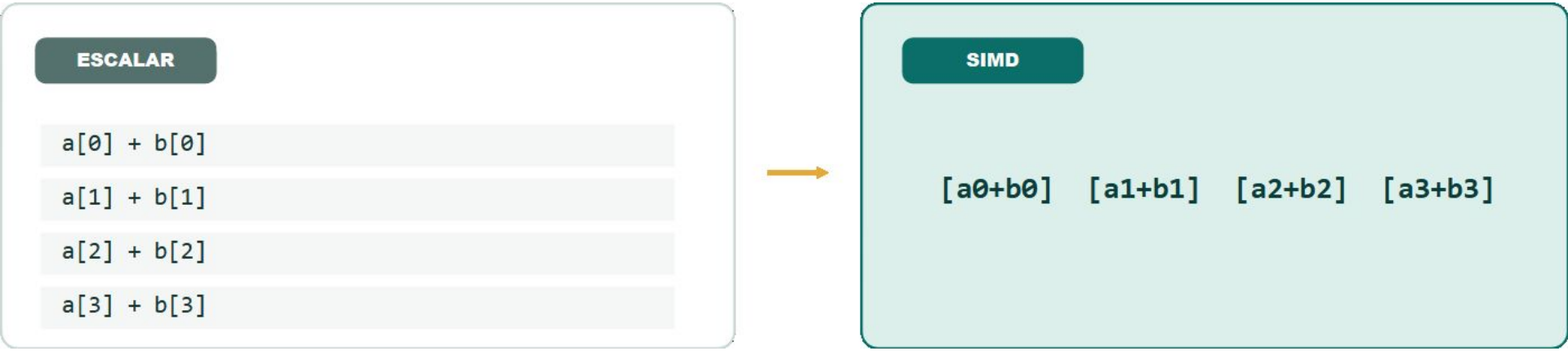
```
while (i + 1 < n) {
    processa(i);
    processa(i + 1);
    i = i + 2;
}
if (i < n) processa(i);
```



Tratar com cuidado overflow do teste, break, continue e return.

Indução

- Vetorização: SIMD depende do alvo.
- Uma operação por lanes; laço regular não implica automaticamente vetorizar com segurança.



Independência
Sem dependências entre iterações/lane.

Suporte
Tipos e operações disponíveis no alvo.

Reduções
Associação e arredondamento podem mudar.

Indução

- A soma ilustra uma indução básica e uma recorrência dependente.

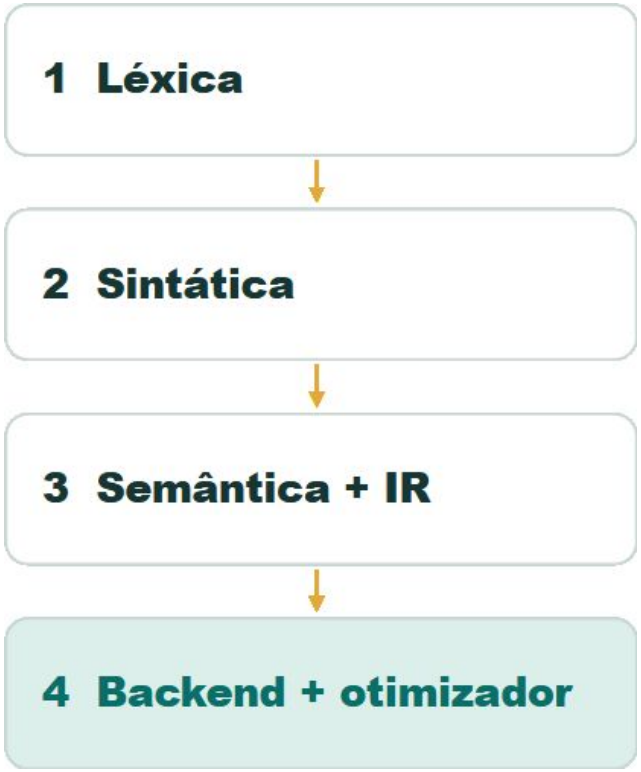
```
int soma(int dados[], int n) {
    int i;
    int total = 0;
    for (i = 0; i < n; i = i + 1) {
        total = total + dados[i];
    }
    return total;
}
```

`i`	Indução básica (+1)
`total`	Redução: depende do valor anterior
`n`	Invariante candidata
`dados[i]`	Acesso regular; limites/alias importam

Sem prova suficiente, manter a soma escalar é uma escolha correta.

Integração do compilador MINIC

- Integração das quatro etapas.
- O resultado de cada etapa é entrada formal da seguinte.



caracteres → tokens	lexemas, posições e erros
tokens → AST	precedência, estrutura e consumo total
AST → símbolos, tipos, 3AC	escopos, tipos, temporários e rótulos
IR → alvo → execução	preservação, chamadas e status coerente

Erros de compilação devem interromper o fluxo antes de otimização ou geração de código.

Integração do compilador MINIC

- Testes de integração e validação.
- O teste diferencial fornece evidência experimental; não prova corretude universal.

Por fase

tokens • AST • semântica • IR

Integração válida

função • vetor • if/for • return

Negativos

léxico • sintaxe • tipos • escopos

Diferencial

mesmo programa em -O0 e -O1

Use mesma entrada; inclua zero/uma iteração, limites, break/continue e regressões.

Integração do compilador MINIC

- Decisões de arquitetura e documentação.
- O projeto deve ser reproduzível por alguém que não conhece decisões tácitas da equipe.

IR	3AC? formato, vetores, chamadas, tipos e rótulos
Alvo	assembly, bytecode, LLVM ou VM; plataforma e ABI
Otimizações	passes, ordem, pré-condições, -O0/-O1
Contratos	diagnósticos, códigos de saída e semântica escolhida
Reprodução	construção, comandos, testes, entradas e resultados

Compare desempenho com método: várias entradas, repetições e mediana; separe compilação de execução.